

Theory Notes — Problem Solving, C Basics & Operators

Unit A — Problem Solving and Logic Development

- 1. Problem Definition
- 2. Problem Analysis
- 3. General Problem-Solving Strategies
- 4. Algorithm Design and Characteristics
- 5. Algorithm Development Techniques
- 6. Flowcharts
- 7. Standard Flowchart Symbols
- 8. Flowchart Designing / Implementation
- 9. Pseudocode
- 10. Algorithm vs Flowchart vs Pseudocode
- 11. Structured vs Unstructured Programming
- 12. Computational Problem Solving

Unit B — Introduction to C Programming

- 13. History and Features of C
- 14. Structure of a C Program
- 15. Compilation and Execution Process
- 16. Basic C Program
- 17. Tokens
- 18. Keywords
- 19. Identifiers
- 20. Variables
- 21. Constants
- 22. Data Types (including Pointers)

Unit C — Operators and Expressions

- 23. Arithmetic Operators
- 24. Relational Operators
- 25. Logical Operators
- 26. Assignment Operators
- 27. Unary Operators
- 28. Increment / Decrement Operators
- 29. Conditional (Ternary) Operator

Unit A · Problem Solving and Logic Development

1. Problem Definition

Problem definition is the process of clearly stating what a program needs to accomplish before any code is written. It identifies exactly what inputs are available, what output is expected, and what rules or constraints apply. A poorly defined problem almost always leads to a wrong or incomplete solution.

EXAMPLE

Problem: "Find the average of three numbers."
Definition — Input: three numbers (say a, b, c); Output: their average; Constraint: numbers can be integers or decimals; Formula: average = (a+b+c)/3.

2. Problem Analysis

Problem analysis goes one step deeper than definition — it breaks the problem into smaller, understandable parts: identifying the *known* values, the *unknown* values to be found, the data needed, and the relationship (formula or logic) that connects them.

EXAMPLE

Problem: "Calculate simple interest."
Analysis — Known: Principal (P), Rate (R), Time (T); Unknown: Simple Interest (SI); Relationship: $SI = (P \times R \times T) / 100$.

3. General Problem-Solving Strategies

Several general strategies help convert a problem into a solvable procedure:

- **Divide and Conquer** — break a big problem into smaller sub-problems, solve each, then combine results.
- **Top-Down (Stepwise Refinement)** — start with a broad solution outline and refine each step into more detail.
- **Bottom-Up** — build small working pieces first, then combine them into the full solution.
- **Trial and Error** — try a possible solution, check the result, and adjust if it fails.
- **Working Backwards** — start from the desired result and work back to figure out the required inputs/steps.

EXAMPLE

Sorting a deck of cards using **divide and conquer**: split the deck into two halves, sort each half separately, then merge the two sorted halves together.

4. Algorithm Design and Characteristics

An **algorithm** is a finite, ordered sequence of well-defined steps that solves a specific problem or performs a specific task. A good algorithm must satisfy these characteristics:

Characteristic	Meaning
Input	Takes zero or more well-defined inputs
Output	Produces at least one well-defined output
Definiteness	Every step must be clear and unambiguous
Finiteness	Must terminate after a finite number of steps
Effectiveness	Each step must be simple enough to be carried out exactly

EXAMPLE – Algorithm to find the largest of two numbers

1. Start
2. Read two numbers A and B
3. If $A > B$, then largest = A; otherwise largest = B
4. Print largest
5. Stop

5. Algorithm Development Techniques

Algorithms are usually developed using structured techniques rather than written all at once:

- **Top-down design** — the problem is expressed as one main task, which is progressively broken into sub-tasks.
- **Stepwise refinement** — each sub-task is refined again and again until it is simple enough to translate directly into code.
- **Modular design** — the solution is organized into independent, reusable modules (functions), each handling one responsibility.

EXAMPLE – Stepwise refinement for "Find a student's grade"

Step 1 (broad): Get marks → calculate percentage → decide grade.
Step 2 (refined): Read marks in 5 subjects → total = sum of marks → percentage = total/5 → if percentage ≥ 90 grade = "A", else if ≥75 grade = "B", else grade = "C".

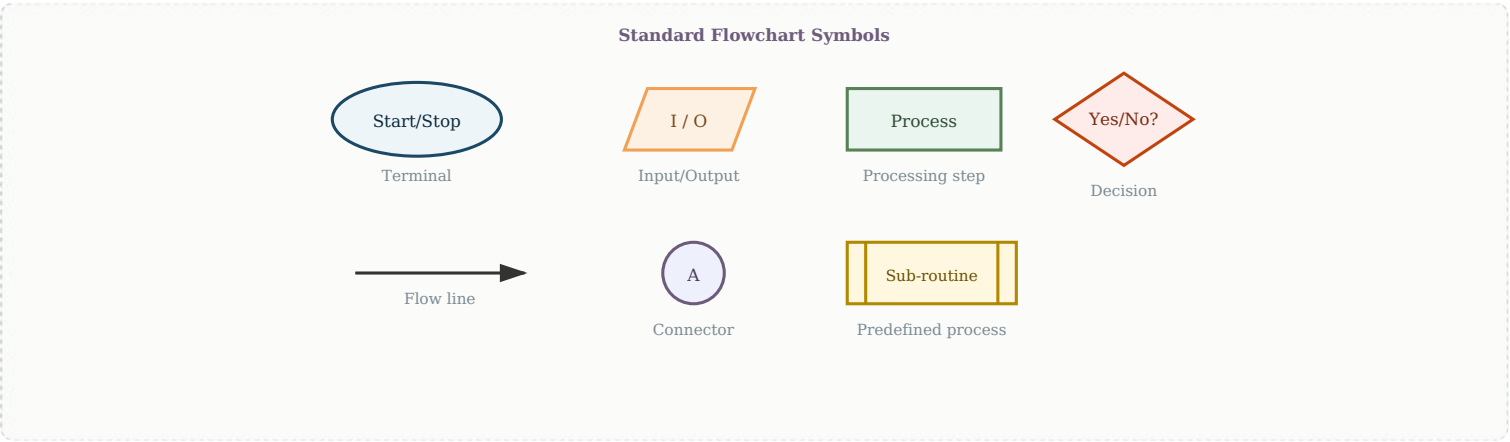
6. Flowcharts

A **flowchart** is a diagrammatic representation of an algorithm. It uses standard geometric shapes connected by arrows to show the sequence of operations, decisions, and the direction of flow of control in a process.

Why use flowcharts? They make the logic of a solution visually easy to follow, help detect errors in logic early, and serve as documentation that is easier for non-programmers to understand than code.

7. Standard Flowchart Symbols

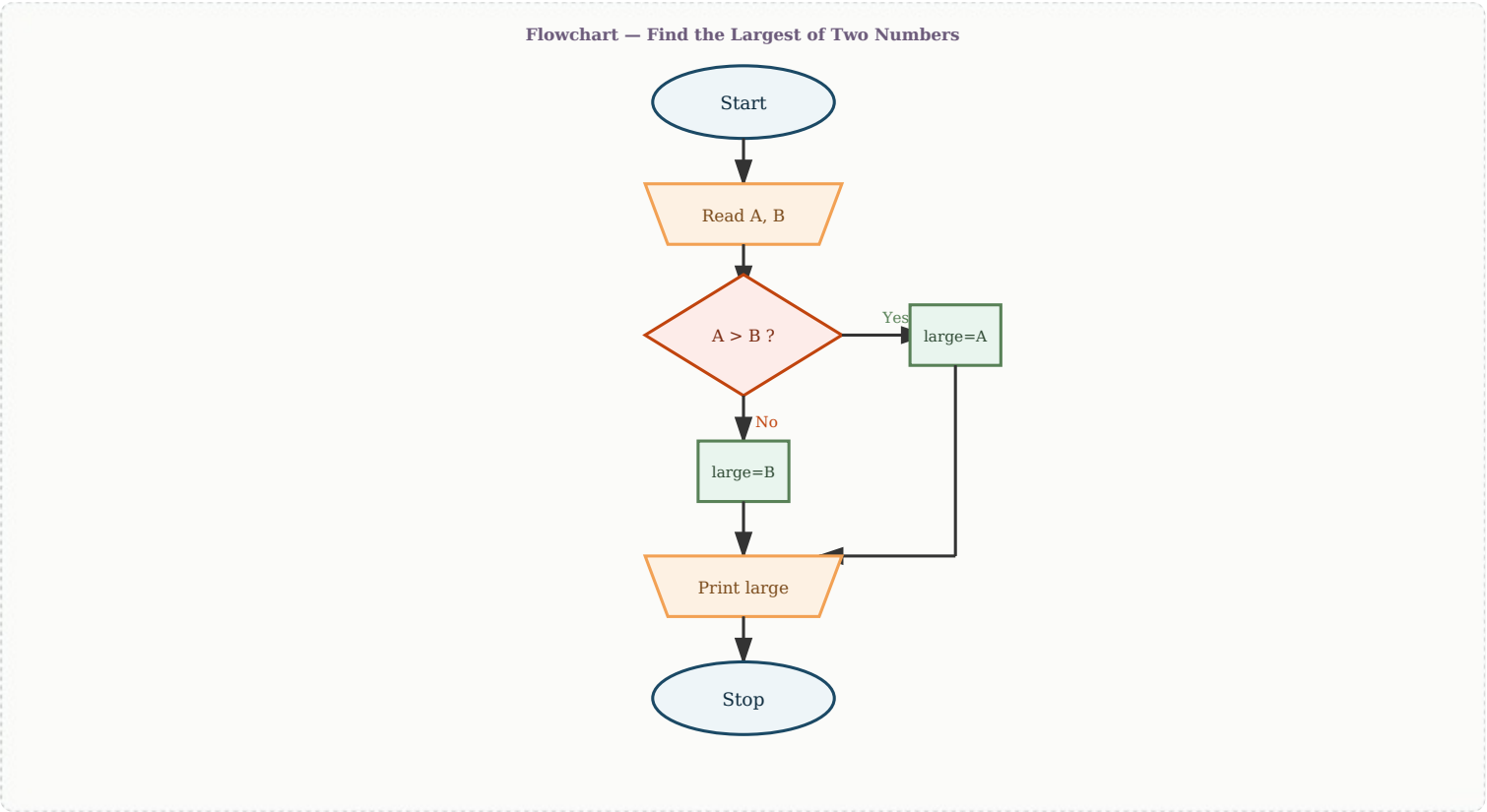
Each flowchart symbol has a specific, standardized meaning:



Symbol	Name	Purpose
Oval	Terminal	Marks the Start or Stop (End) of a flowchart
Parallelogram	Input/Output	Represents reading input or displaying output
Rectangle	Process	Represents a calculation or processing step
Diamond	Decision	Represents a yes/no or true/false decision point
Arrow	Flow line	Shows the direction of program flow
Circle	Connector	Joins different parts of a flowchart, especially across pages
Rectangle with double bars	Predefined process	Represents a call to a sub-routine or function

8. Flowchart Designing / Implementation

Designing a flowchart means translating an algorithm's steps directly into the matching symbols and connecting them in the correct order with flow-line arrows.



For A = 7, B = 4: the decision box "A > B?" evaluates to Yes, so large = A = 7 is printed.

9. Pseudocode

Pseudocode is an informal, plain-language way of describing an algorithm's logic using structured, code-like statements, without following the strict syntax rules of any real programming language. It is easier to write and read than actual code, yet more precise than a plain paragraph.

EXAMPLE – Pseudocode for finding the largest of two numbers

```
BEGIN
  READ A, B
  IF A > B THEN
    large = A
  ELSE
    large = B
  ENDIF
  PRINT large
END
```

10. Algorithm vs Flowchart vs Pseudocode

Aspect	Algorithm	Flowchart	Pseudocode
Form	Numbered plain-language steps	Graphical, symbol-based	Structured, code-like text
Readability	Easy but less visual	Very easy to visualize flow	Close to real code, precise
Best for	Describing overall logic quickly	Showing flow of control visually	Bridging logic and actual coding
Time to create	Fast	Slower for complex logic	Moderate

11. Structured vs Unstructured Programming

Structured programming builds a program using only three control structures — *sequence*, *selection* (if/else), and *iteration* (loops) — executed top-to-bottom without arbitrary jumps. **Unstructured programming** relies heavily on unconditional jumps (like goto) that can transfer control anywhere in the program, often making the logic tangled and hard to follow.

Structured Programming	Unstructured Programming
Uses sequence, selection, iteration	Relies on goto jumps
Easy to read, debug, and maintain	Hard to trace and debug ("spaghetti code")
Broken into modules/functions	Often one long, unbroken block

EXAMPLE

Structured: if (a > b) { printf("A is larger"); }
Unstructured (avoid): using goto label; to jump around a program instead of a clean if statement.

12. Computational Problem Solving

Computational problem solving is the complete cycle a programmer follows to turn a real-world problem into a working program:

1. **Understand** the problem (definition & analysis)
2. **Design** a solution (algorithm, flowchart, or pseudocode)
3. **Code** the solution in a programming language
4. **Test** the program with sample data
5. **Debug** and fix any errors found
6. **Maintain** the program as requirements change over time

Key idea: Skipping the design stage and jumping straight to code is the most common cause of buggy, hard-to-fix programs.

Unit B · Introduction to C Programming

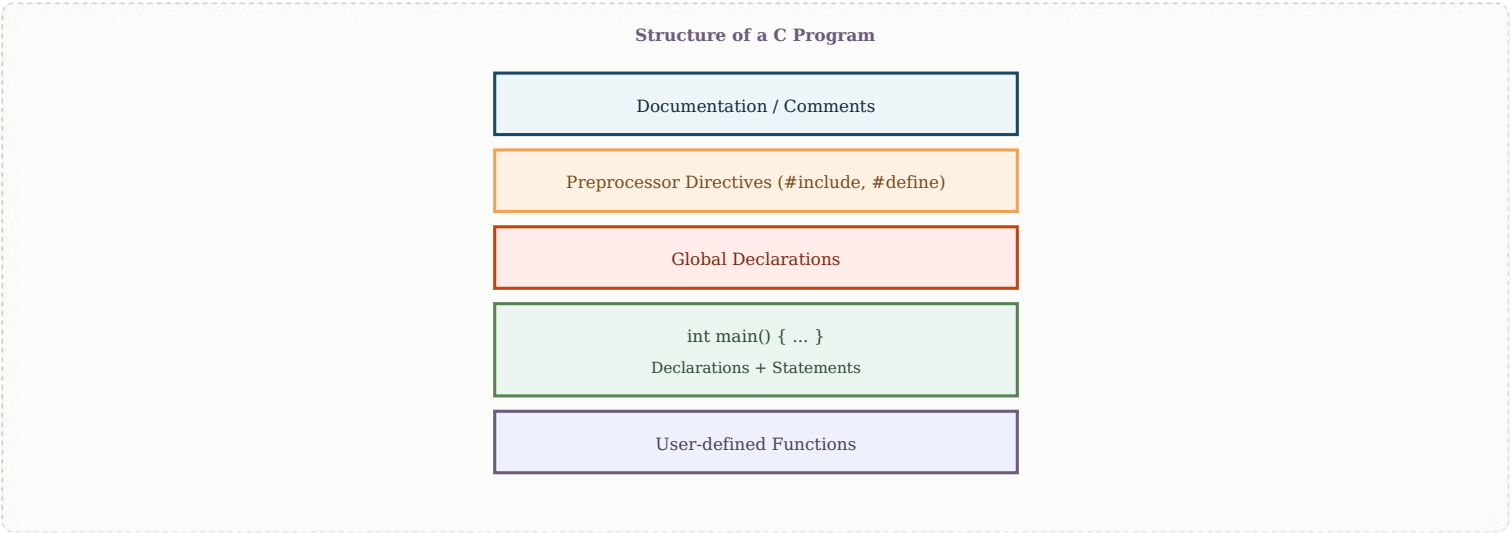
13. History and Features of C

C was developed by **Dennis Ritchie** at **Bell Labs** in **1972**, evolving from earlier languages called **B** and **BCPL**. It was originally created to rewrite the UNIX operating system and has since become one of the most influential programming languages in history.

Feature	Description
Mid-level language	Combines features of high-level and low-level (assembly-like) languages
Structured	Programs are built from functions and blocks
Portable	C code can run on different machines with little or no change
Fast and efficient	Compiles to efficient machine code, close to hardware
Rich library	Provides many built-in functions (stdio.h, math.h, etc.)
Pointer support	Allows direct memory access and manipulation

14. Structure of a C Program

Every C program generally follows this layout:



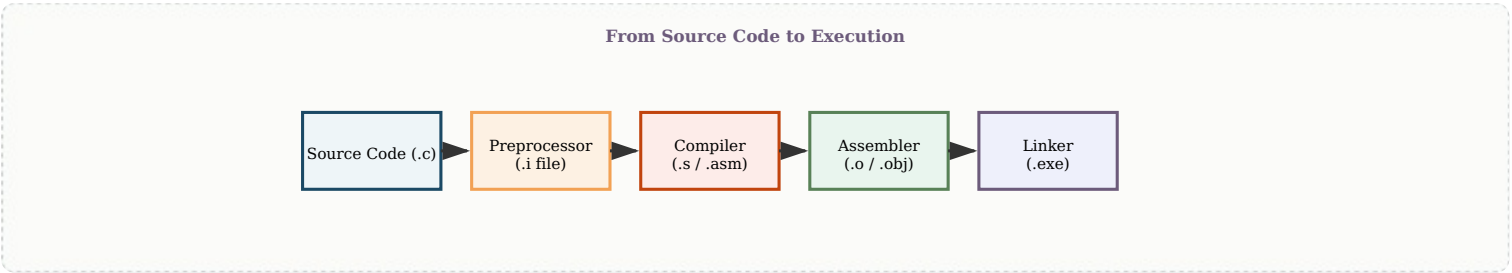
EXAMPLE

```
/* Documentation: adds two numbers */
#include <stdio.h>           // Preprocessor directive

int main() {                // main function
    int a = 5, b = 3;        // declarations
    printf("Sum = %d", a+b); // statement
    return 0;
}
```

15. Compilation and Execution Process

Turning C source code into a running program involves several stages:



- Preprocessing** — handles directives like `#include` and `#define`, expanding them into the source.
- Compilation** — translates the preprocessed code into assembly language.
- Assembly** — converts assembly code into machine-readable object code.
- Linking** — combines object code with library code to produce the final executable file.
- Execution / Loading** — the operating system loads the executable into memory and runs it.

16. Basic C Program

EXAMPLE – A minimal C program

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

This program includes the standard I/O header, defines the mandatory `main()` function where execution begins, prints a message, and returns 0 to signal successful completion to the operating system.

17. Tokens

A **token** is the smallest individual unit in a C program that the compiler can recognize. C source code is broken into a stream of tokens before it is compiled.

EXAMPLE – Tokens in one line of code

```
int total = 10 + 5;
```

Token	Category
int	Keyword
total	Identifier
=	Operator
10, 5	Constants
+	Operator
;	Special symbol

18. Keywords

Keywords are reserved words that have a predefined, fixed meaning in C. They cannot be used as variable names or other identifiers. C has 32 standard keywords.

EXAMPLE – Some common keywords

int, float, char, if, else, for, while, return, void, const, struct, break

19. Identifiers

Identifiers are names given by the programmer to variables, functions, arrays, and other program elements. Rules: must start with a letter or underscore, may contain letters/digits/underscores, cannot be a keyword, and are case-sensitive.

EXAMPLE

Valid: studentAge, _percentage, total_marks

Invalid: 2ndYear (starts with a digit), total-fee (hyphen not allowed)

20. Variables

A **variable** is a named storage location in memory whose value can change during program execution. It must be declared with a data type before use.

EXAMPLE

```
int age = 20;      // variable 'age' stores an integer
age = 21;          // value can be changed later
```

21. Constants

A **constant** is a fixed value that does not change during program execution. C supports integer, floating-point, character, and string constants, and they can be defined using the `#define` preprocessor directive or the `const` keyword.

EXAMPLE

```
#define PI 3.14159      // macro constant
const int MAX = 100;    // typed constant
```

22. Data Types (including Pointers)

A **data type** specifies what kind of value a variable can hold and how much memory it occupies. C groups data types into three broad categories:

Category	Examples
Basic (primary)	int, float, double, char
Derived	Arrays, Pointers, Functions
User-defined	struct, union, enum

Pointers as a data type: a pointer does not store an ordinary value — it stores a memory **address**. It is declared with the type of data it points to (e.g. `int *p` is "pointer to int"), making it a distinct derived data type in its own right.

EXAMPLE

```
int num = 10;
int *ptr = &num;    // ptr's type is "pointer to int"
printf("%d", *ptr); // dereferencing prints 10
```

Unit C · Operators and Expressions

23. Arithmetic Operators

Used to perform mathematical calculations on numeric operands.

Operator	Meaning	Example (a=10, b=3)
+	Addition	a+b → 13
-	Subtraction	a-b → 7
*	Multiplication	a*b → 30
/	Division	a/b → 3 (integer division)
%	Modulus (remainder)	a%b → 1

24. Relational Operators

Used to compare two values; the result is always 1 (true) or 0 (false).

Operator	Meaning	Example (a=5, b=8)
>	Greater than	a>b → 0
<	Less than	a<b → 1
>=	Greater than or equal to	a>=b → 0
<=	Less than or equal to	a<=b → 1
==	Equal to	a==b → 0
!=	Not equal to	a!=b → 1

25. Logical Operators

Used to combine or invert conditions, mainly inside if statements and loops.

Operator	Meaning	Example
&&	Logical AND — true only if both conditions are true	(age>=18 && hasID)
	Logical OR — true if at least one condition is true	(isMember isGuest)
!	Logical NOT — reverses a condition's result	!isCitizen

26. Assignment Operators

Used to assign a value to a variable. C also provides compound assignment operators that combine an arithmetic operation with assignment.

Operator	Meaning	Equivalent To
=	Simple assignment	a = 5;
+=	Add and assign	a = a + 5;
-=	Subtract and assign	a = a - 5;
*=	Multiply and assign	a = a * 5;
/=	Divide and assign	a = a / 5;
%=	Modulus and assign	a = a % 5;

27. Unary Operators

Unary operators act on a **single** operand.

Operator	Meaning	Example
-	Unary minus (negation)	-a
!	Logical NOT	!flag
++	Increment by 1	a++ or ++a
--	Decrement by 1	a-- or --a
sizeof	Size of a type/variable in bytes	sizeof(int)
&	Address-of	&a

*	Dereference (pointer)	*ptr
---	-----------------------	------

28. Increment / Decrement Operators

These special unary operators increase or decrease a variable's value by exactly 1. They come in two forms:

Form	Behavior
Pre-increment ++a	Value is increased first , then used in the expression
Post-increment a++	Current value is used in the expression first , then increased
Pre-decrement --a	Value is decreased first, then used
Post-decrement a--	Current value used first, then decreased

EXAMPLE (a = 5)

printf("%d", a++); prints **5**, and afterward a becomes 6.
printf("%d", ++a); increases a to 6 first, and prints **6**.

29. Conditional (Ternary) Operator

The **ternary operator** `? :` is C's only operator that takes three operands. It offers a compact, one-line alternative to a simple if-else statement.

SYNTAX

result = (condition) ? value_if_true : value_if_false;

EXAMPLE

```
int a = 15, b = 9;  
int max = (a > b) ? a : b;    // max becomes 15
```